

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа
“ДЗ4: Технический дизайн микросервисной архитектуры”

Выполнил:

Березина Софья

Группа

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить существующее монолитное бэкенд-приложение (платформа для фитнес-тренировок и здоровья) на микросервисы, придерживаясь концепции database-per-service. Необходимо спроектировать микросервисную архитектуру, определить способы взаимодействия сервисов, описать новые эндпоинты для межсервисного взаимодействия в формате OpenAPI, а также задокументировать возможные ошибки. Результат должен содержать конкретные шаги для декомпозиции монолита.

Ход работы

1. Выделение микросервисов

Анализ монолитного приложения показал, что его можно разделить на шесть независимых сервисов, каждый из которых отвечает за свою предметную область:

Микросервис	Ответственность	Таблицы (БД)
Auth Service	Аутентификация, выдача и валидация JWT-токенов, ревокация	revoked_tokens, refresh_tokens
User Service	Управление пользователями, профили, роли	users, user_profiles
Workout Service	Управление шаблонами тренировок и упражнений	workouts, exercises
UserWorkouts Service	Назначение тренировок пользователям, отслеживание выполнения	user_workouts, user_exercises, sessions
Progress Service	Агрегирование прогресса пользователей, вычисление метрик	user_progress_entries, aggregates
Blog Service	Публикация, чтение и управление блог-постами	posts, comments, tags

Обоснование выделения:

- Auth Service - независимый домен аутентификации, может быть переиспользован другими сервисами.
- User Service - базовая информация о пользователях, необходима всем остальным сервисам.
- Workout Service - основная предметная область (тренировки и упражнения).
- UserWorkouts Service - логика выполнения тренировок вынесена отдельно, чтобы масштабировать независимо при высокой нагрузке.
- Progress Service - аналитика и статистика, требует отдельной базы для агрегации.
- Blog Service - контентная часть, может кэшироваться и масштабироваться отдельно.

2. Взаимодействие микросервисов

Вызывающий сервис	Целевой сервис	Метод	Внутренний эндпоинт	Когда вызывается
Любой сервис	Auth Service	POST	/internal/auth/validate	Проверка JWT токена
Workout Service	User Service	GET	/internal/users/{id}	При создании тренировки (проверка автора)
UserWorkout Service	Auth Service	GET	/internal/users/{id}/exists	При назначении тренировки (проверка пользователя)
UserWorkout Service	Workout Service	GET	/internal/workouts/{id}	При назначении тренировки (проверка шаблона)

Progress Service	User Workout Service	GET	/internal/user-workouts/stats	При расчёте статистики
Blog Service	Auth Service	GET	/internal/users/{id}/exists	При создании статьи (проверка автора)

3. Эндпоинты для межсервисного взаимодействия (OpenAPI)

Сервисы вызывают друг друга по внутренним эндпоинтам, недоступным извне.

3.1. Auth Service (внутренние эндпоинты)

```

openapi: 3.0.0
info:
  title: Auth Service Internal API
  version: 1.0.0
paths:
  /internal/auth/validate:
    post:
      summary: Проверить валидность JWT токена
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                token:
                  type: string
      responses:
        '200':
          description: Токен валиден
          content:
            application/json:
              schema:
                type: object
                properties:
                  valid:
                    type: boolean
                  user_id:
                    type: string
                  email:
                    type: string
                  role:
                    type: string
        '401':
          description: Токен невалиден или истёк

  /internal/users/{id}/exists:
    get:

```

summary: Проверить существование пользователя
parameters:
- **name:** id
 in: path
 required: true
 schema:
 type: string
responses:
'200':
 description: Пользователь существует
 content:
 application/json:
 schema:
 type: object
 properties:
 exists:
 type: boolean
 email:
 type: string
 role:
 type: string
'404':
 description: Пользователь не найден

3.2. User Service (внутренние эндпоинты)

paths:
/internal/users/{id}:
get:
 summary: Получить профиль пользователя по ID
 parameters:
 - **name:** id
 in: path
 required: true
 schema:
 type: string
 responses:
 '200':
 description: Успешно
 content:
 application/json:
 schema:
 type: object
 properties:
 id:
 type: string
 full_name:
 type: string
 fitness_level:
 type: string
 height_cm:
 type: number
 weight_kg:
 type: number
 '404':
 description: Профиль не найден

3.3. Workout Service (внутренние эндпоинты)

```
paths:
  /internal/workouts/{id}:
    get:
      summary: Получить тренировку по ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: Тренировка найдена
          content:
            application/json:
              schema:
                type: object
                properties:
                  id:
                    type: string
                  title:
                    type: string
                  duration_min:
                    type: integer
                  exercises:
                    type: array
        '404':
          description: Тренировка не найдена
```

3.4. UserWorkout Service (внутренние эндпоинты)

```
paths:
  /internal/user-workouts/stats:
    get:
      summary: Получить статистику тренировок пользователя
      parameters:
        - name: user_id
          in: query
          required: true
          schema:
            type: string
      responses:
        '200':
          description: Статистика
          content:
            application/json:
              schema:
                type: object
                properties:
                  total_workouts:
                    type: integer
                  completed_workouts:
                    type: integer
                  average_rating:
                    type: number
```

4. Варианты запросов и ответов, возможные ошибки

Пример запроса от Workout Service к User Service:

GET /internal/users/123

Ответ (200):

```
{  
  "id": "123",  
  "full_name": "Иван Иванов",  
  "fitness_level": "intermediate",  
  "height_cm": 175,  
  "weight_kg": 70.5  
}
```

5. План миграции монолита в микросервисы

Шаг 1. Подготовка

- Создать отдельные базы данных для каждого сервиса.
- Развернуть API Gateway.

Шаг 2. Выделение Auth Service

- Скопировать сущность User в новый сервис.
- Перенести эндпоинты /auth/* и /users/me/password.
- Настроить общий JWT-секрет для всех сервисов.
- Добавить внутренние эндпоинты /internal/auth/validate и /internal/users/{id}/exists.

Шаг 3. Выделение User Service

- Скопировать сущность UserProfile в новый сервис.
- Перенести эндпоинты /users/me/profile.
- Добавить внутренний эндпоинт /internal/users/{id}.

Шаг 4. Выделение Workout Service

- Скопировать сущности Workout, Exercise.
- Перенести эндпоинты /workouts/*, /admin/workouts/*.
- Добавить внутренний эндпоинт /internal/workouts/{id}.

Шаг 5. Выделение UserWorkout Service

- Скопировать сущность UserWorkout.
- Перенести эндпоинты /user/workouts/*.

- Настроить вызовы к Auth Service, User Service и Workout Service.

Шаг 6. Выделение Progress Service

- Скопировать сущность UserProgress.
- Перенести эндпоинты /progress/*.

Шаг 7. Выделение Blog Service

- Скопировать сущность BlogPost.
- Перенести эндпоинты /blog/*, /admin/blog/*.

Шаг 8. Настройка API Gateway

- Настроить маршрутизацию всех публичных эндпоинтов.
- Настроить проверку JWT на уровне Gateway (вызов Auth Service).

Шаг 9. Тестирование

- Проверить работу всех эндпоинтов через Postman.
- Убедиться, что межсервисные вызовы работают корректно.

Вывод

В результате работы спроектирована микросервисная архитектура для платформы фитнес-тренировок и здоровья. Выделены шесть сервисов (Auth, User, Workout, UserWorkout, Progress, Blog), каждый со своей базой данных (database-per-service). Auth Service выделен отдельно, так как он обеспечивает сквозную аутентификацию для всех остальных сервисов. Описаны внутренние эндпоинты для синхронного взаимодействия в формате OpenAPI. Предложены конкретные шаги по декомпозиции существующего монолита. Взаимодействие между сервисами осуществляется синхронно через HTTP, что достаточно для учебного проекта и упрощает реализацию.